```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
```

```
from google.colab import files

uploaded = files.upload()
```

Choose Files Food_Ingre…atabase.csv

Food_Ingredient_Intelligence_Database.csv(text/csv) - 4024274 bytes, last modified: 2/12/2026 - 100% done

```
Saving Food_Ingredient_Intelligence_Database.csv to Food_Ingredient_Intelligence_Database.csv
```

```
serving_size_np = y.to_numpy()
print("Mean:", np.mean(serving_size_np))
print("Median:", np.median(serving_size_np))
print("Standard Deviation:", np.std(serving_size_np))
```

```
Mean: 83.45058566886877
Median: 47.0
Standard Deviation: 131.9291813463961
```

```
min_val = np.min(serving_size_np)
max_val = np.max(serving_size_np)

serving_size_normalized = (serving_size_np - min_val) / (max_val - min_val)
serving_size_normalized
```

```
array([0.0037963 , 0.00240741, 0.00240741, ..., 0.00277778, 0.00277778,
       0.00361111])
```

```
df.head()
```

	fdc_id	description	brand_owner	brand_name	ingredients	serving_size	serving_size_unit	food_category	search_category
0	2461174	WHEAT BREAD, WHEAT	Bimbo Bakeries USA, Inc.	SELECT	ENRICHED WHEAT FLOUR [FLOUR, MALTED BARLEY FLO...	41.0	g	Breads & Buns	wheat
1	2162090	WHEAT BREAD, WHEAT	The Moran Group Incorporated	SAVE A LOT	ENRICHED WHEAT FLOUR (FLOUR, MALTED BARLEY	26.0	g	Breads & Buns	wheat

Next steps: Generate code with df New interactive sheet

```
print("Shape:", df.shape)
print("Columns:", df.columns)
print("Missing Values:\n", df.isnull().sum())
```

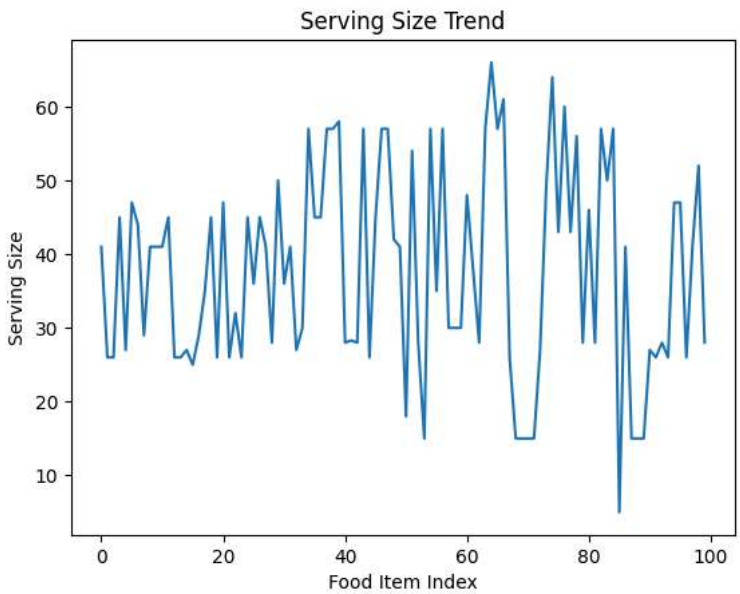
```
Shape: (12321, 9)
Columns: Index(['fdc_id', 'description', 'brand_owner', 'brand_name', 'ingredients',
       'serving_size', 'serving_size_unit', 'food_category',
       'search_category'],
      dtype='object')
Missing Values:
 fdc_id                 0
description            0
```

```
brand_owner           83
brand_name           556
ingredients          188
serving_size           0
serving_size_unit      0
food_category          0
search_category        0
dtype: int64
```

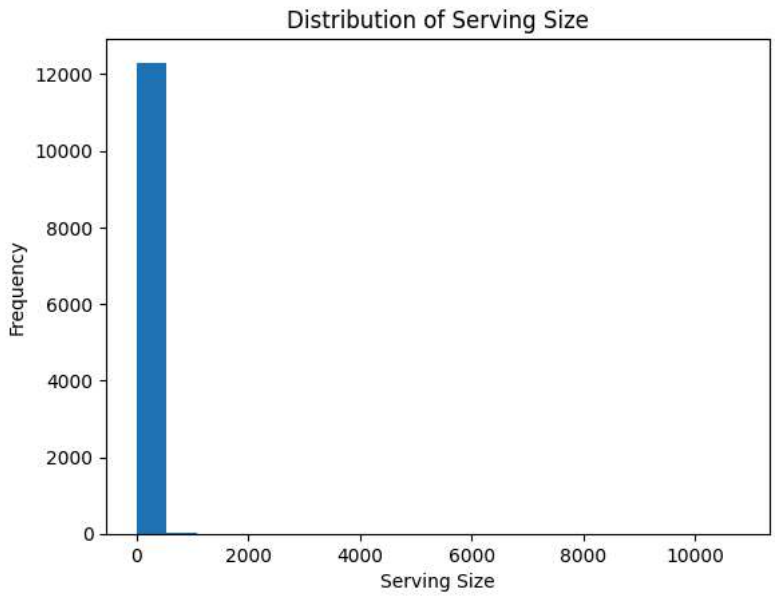
```
def performance_label(size):
    if size >= 200:
        return "High"
    elif size >= 100:
        return "Medium"
    else:
        return "Low"

df['Performance'] = df['serving_size'].apply(performance_label)

df[['serving_size', 'Performance']].head()
```

	serving_size	Performance
0	41.0	Low
1	26.0	Low
2	26.0	Low
3	45.0	Low
4	27.0	Low

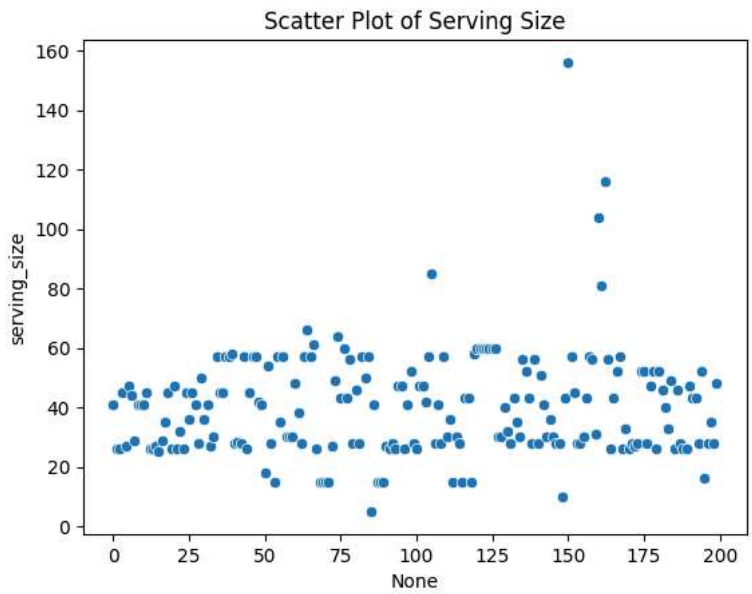
```
plt.plot(df.index[:100], df['serving_size'][:100])
plt.xlabel("Food Item Index")
plt.ylabel("Serving Size")
plt.title("Serving Size Trend")
plt.show()
```



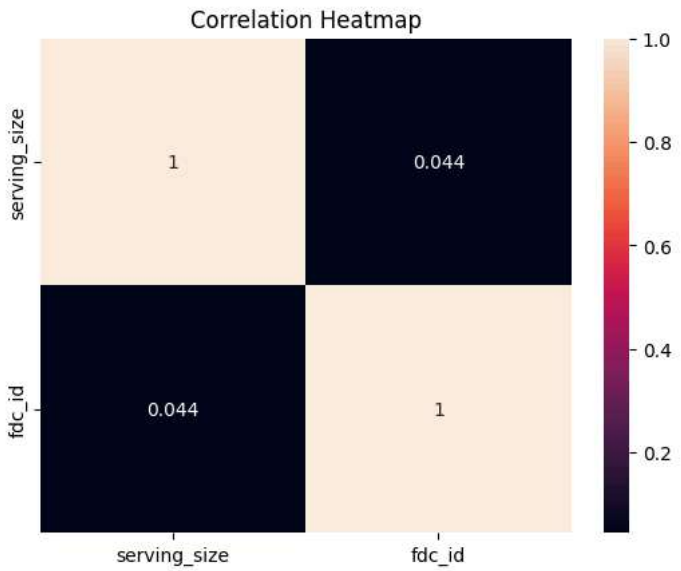

```
plt.hist(df['serving_size'], bins=20)
plt.xlabel("Serving Size")
plt.ylabel("Frequency")
plt.title("Distribution of Serving Size")
plt.show()
```

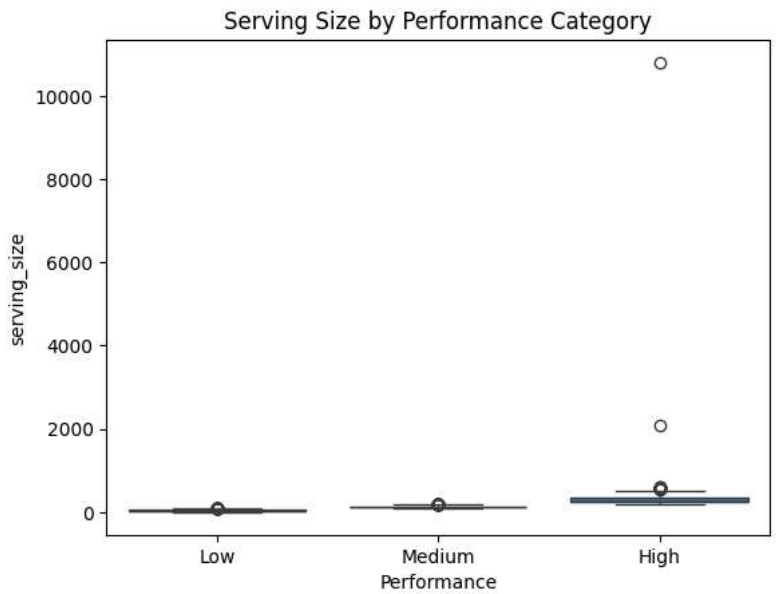



```
sns.scatterplot(x=df.index[:200], y=df['serving_size'][:200])
plt.title("Scatter Plot of Serving Size")
plt.show()
```




```
corr = df[['serving_size', 'fdc_id']].corr()

sns.heatmap(corr, annot=True)
plt.title("Correlation Heatmap")
plt.show()
```




```
sns.boxplot(x='Performance', y='serving_size', data=df)
plt.title("Serving Size by Performance Category")
plt.show()
```